arXiv:2611.13293v1 [cs.DC] 9 Nov 2026

DECODE TOKEN PARALLELISM: MEMORY-EFFICIENT DISTRIBUTED ATTENTION PARALLELISM FOR SCALABLE LLM DECODING

Susav Shrestha [1] **Brad Settlemyer** [1] **Narasimha Reddy** [2]

ABSTRACT

Serving large language models at high concurrency with long contexts requires KV-cache capacity and attention bandwidth that grow with the decode pool. Existing parallelization strategies trade one against the other: data parallelism replicates model weights, limiting KV capacity; tensor parallelism stops scaling attention at the model's KV-head count and pushes dense collectives across nodes; and decode context parallelism (DCP) recovers capacity by sharding the sequence but adds cross-node aggregation of partial attention results. We observe that during decode, dense layers and attention scale differently: dense layers amortize weight reads across the batch and run efficiently within a single NVLink domain, whereas attention cost grows with every active request's KV cache. We present **Decode Token Parallelism (TKNP)**, a decode-pool architecture that decouples weight placement from attention ownership. TKNP distributes a single model copy across nodes in balanced layer blocks. The active node holding a block executes its projections and MLP for the full batch using intra-node tensor parallelism, while every node computes attention for its own requests against locally resident KV cache. This extends request-local attention across nodes without replicating weights: only current-token activations cross the network, dense all-reduces remain within NVLink, and attention requires no cross-node merging of partial results. Implemented in vLLM and evaluated on Llama 3.1 405B and Nemotron-4 340B on up to 64 H100 GPUs, TKNP improves completion throughput for long-context, high-concurrency workloads by **up to 1.7× over DCP** and by **up to 2.06× over data parallelism** at matched GPU count.

1 INTRODUCTION

Large language model serving must sustain high throughput while responding promptly to many concurrent requests. Modern inference engines use continuous batching to amortize model-weight reads across requests and admit new work as earlier requests finish (Yu et al., 2022; Kwon et al., 2023; Zheng et al., 2024). For models with hundreds of billions of parameters, however, the memory required to store model weights leaves limited space for long-context KV caches. Increasing the batch improves the efficiency of dense computation, but also increases KV-cache demand, preventing additional requests from becoming resident.

The root of this tension is that dense layers and attention scale differently during decode. MLPs and projections process only newly generated tokens, allowing weight reads to be shared across the batch, whereas attention reads the accumulated KV cache of every active request. As Figure 1 shows, attention approaches 70% of single-layer decode latency for Llama 3.1 405B at batch 128 with 32K context while MLP and projection latency remain relatively stable. Scaling such workloads therefore requires additional attention bandwidth and KV capacity while preserving efficient dense execution. On multi-node GPU clusters, this must

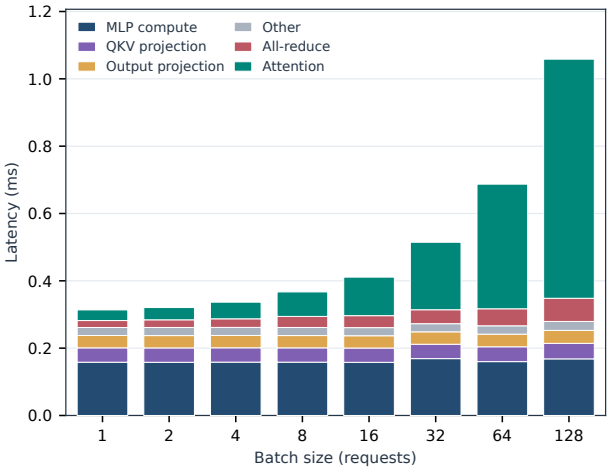


Figure 1. **Decode-stage latency breakdown for a single decoder layer of Llama 3.1 405B** on eight H100 GPUs (TP8) with 32K context. Attention grows with batch size, accounting for nearly 70% of summed component latency at batch 128.

[1]NVIDIA, Santa Clara, United States [2]Texas A&M University, College Station, United States. Correspondence to: Susav Shrestha <sshrestha@nvidia.com>.

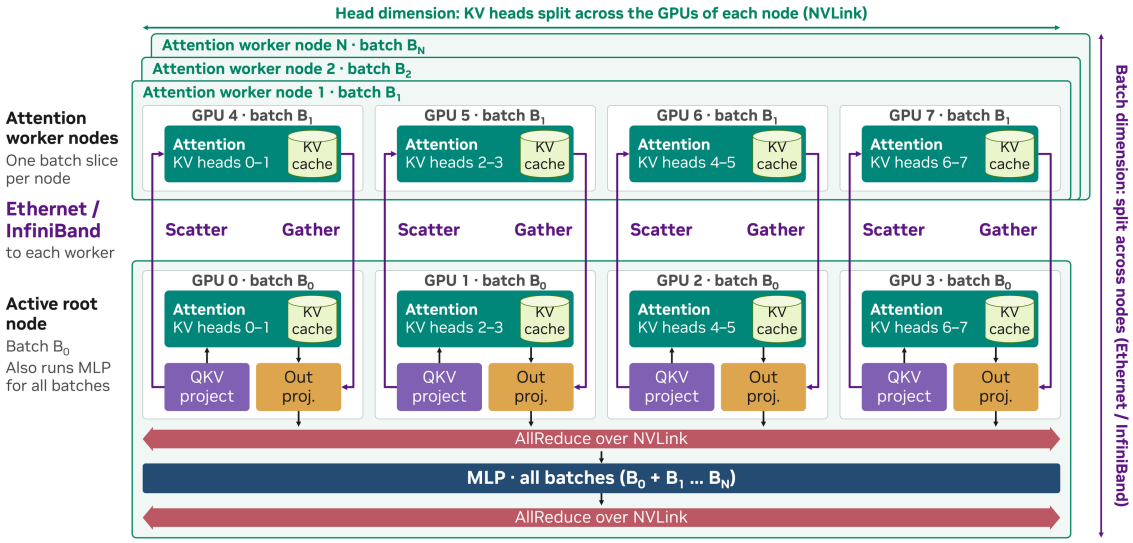


Figure 2. **Token Parallel architecture within a layer block.** TP partitions KV heads within each node; TKNP partitions requests across nodes. The active root executes QKV and output projections and the MLP for the full batch, while all nodes compute attention for their own requests using local KV cache. Scatter/gather exchanges current-token activations over Ethernet or InfiniBand; dense all-reduces remain within NVLink. Four GPUs per node are shown for illustration. Weight storage on attention workers is omitted; the active root changes with the block.

also account for the interconnect hierarchy: GPUs communicate through high-bandwidth NVLink within a node, while cross-node communication uses Ethernet or InfiniBand.

Existing parallelization strategies address these requirements through different tradeoffs. Data parallelism (DP) keeps execution local but replicates model weights, shrinking aggregate KV capacity. Tensor parallelism (TP) replicates KV heads once its degree exceeds the KV-head count (Shoeybi et al., 2020; Ainslie et al., 2023) and moves dense all-reduces onto the inter-node network when it spans nodes. Decode context parallelism (DCP) shards KV cache beyond this limit but must combine partial attention results across ranks. Pipeline parallelism (PP) avoids replication but confines each layer's attention to one stage and repeats dense weight reads across microbatches (Huang et al., 2019; Narayanan et al., 2019). Data-parallel attention for MoE serving (SGLang, 2024) keeps attention local to each request's KV owner but replicates attention weights on every rank, while attention–FFN disaggregation (Chen et al., 2025; Zhu et al., 2025) leaves the weight pool's spare memory unused for KV cache. These tradeoffs motivate separating the placement of model weights from the ownership of attention and KV cache.

We introduce **Decode Token Parallelism (TKNP)**, a decode-pool architecture for prefill–decode disaggregated serving that **decouples where model weights reside from where attention is computed**. TKNP distributes one model copy in stationary layer blocks, balancing weight storage to leave similar KV-cache budgets across nodes. Each layer's projections and MLP execute on its weight-owning active root node using local TP, while all nodes compute attention for their assigned requests. As shown in Figure 2, attention scales across heads within nodes and requests across nodes, with only current-token activations crossing the network, dense all-reduces staying within NVLink, and no cross-node merging of partial attention results.

We implement TKNP in vLLM and evaluate it against DP, TP, PP, and DCP baselines on Llama 3.1 405B (Grattafiori et al., 2024) and Nemotron-4 340B (Nvidia et al., 2024) using up to 64 H100 GPUs. Our contributions are:

- We separate attention ownership from weight placement, enabling **efficient attention scaling** across nodes without replicating model weights.
- We balance weight placement so that **every node contributes approximately equal KV capacity**, matching sequence-sharded layouts and providing up to **5.0× that of DP** on 64 GPUs.
- We propose dual-active-root microbatching to **overlap dense computation and attention**, improving throughput by up to 11.1%.
- We demonstrate **completion-throughput gains of up to 2.06× over DP and 1.7× over DCP** at matched GPU counts and workloads.

2 BACKGROUND

LLM inference consists of two distinct stages: prefill processes the input sequence in parallel, while decode generates subsequent tokens autoregressively. Prefill is typically compute-intensive, whereas long-context decoding places increasing demands on memory bandwidth as attention repeatedly reads the growing key-value (KV) cache (Lin et al., 2024; Wu et al., 2024a; Dao et al., 2022; Ye et al., 2025).

The KV cache avoids recomputing keys and values, but its size grows linearly with batch size, sequence length, layer count, and KV-head count (Kwon et al., 2023). Even with cache-reduction techniques (Shazeer, 2019; Ainslie et al., 2023), long contexts and high concurrency place substantial demands on memory capacity and bandwidth. More broadly, prior GPU-centric systems improve memory efficiency by coordinating data placement and overlapping data movement with computation (Shrestha et al., 2024; 2025a).

2.1 Inference Optimizations and Disaggregation

Serving systems improve utilization through iteration-level scheduling (Yu et al., 2022; Wu et al., 2024b; Agrawal et al., 2024) and paged KV-cache allocation (Kwon et al., 2023), while speculative decoding reduces serial generation steps (Leviathan et al., 2023; Chen et al., 2023; Gautam et al., 2025; Cai et al., 2024; Li et al., 2025). These optimizations operate within a serving configuration; prefill–decode disaggregation additionally separates the two stages into resource pools that can be provisioned and scaled independently (Zhong et al., 2024; Hu et al., 2025; Mitra et al., 2025; Patel et al., 2024). This separation reduces interference between compute-intensive prefill and iterative decoding, while allowing each pool to use different batching and parallelization strategies. TKNP targets execution within the decode pool. The prefill service computes each request's KV cache and transfers it directly to the decode node that will own that request, in TKNP's head layout.

Disaggregation can also separate operators within decoding. Lamina offloads attention and KV cache to memory-oriented devices (Chen et al., 2025), and Hetis distributes attention across heterogeneous GPUs (Mo et al., 2025). MegaScale-Infer separates attention and expert FFNs for independent scaling in MoE serving (Zhu et al., 2025). KernelFlume targets elastic scaling through expanding attention nodes (Xiang et al., 2026). These designs place weights and attention on separate devices, so weight-holding devices either contribute no KV capacity or have smaller KV budgets than pure attention devices. TKNP instead balances layer-block weights across the same nodes that own request KV cache, so every node contributes approximately equal KV capacity and attention bandwidth, and serves as the dense root for its own blocks.

2.2 Parallelism Strategies for Inference

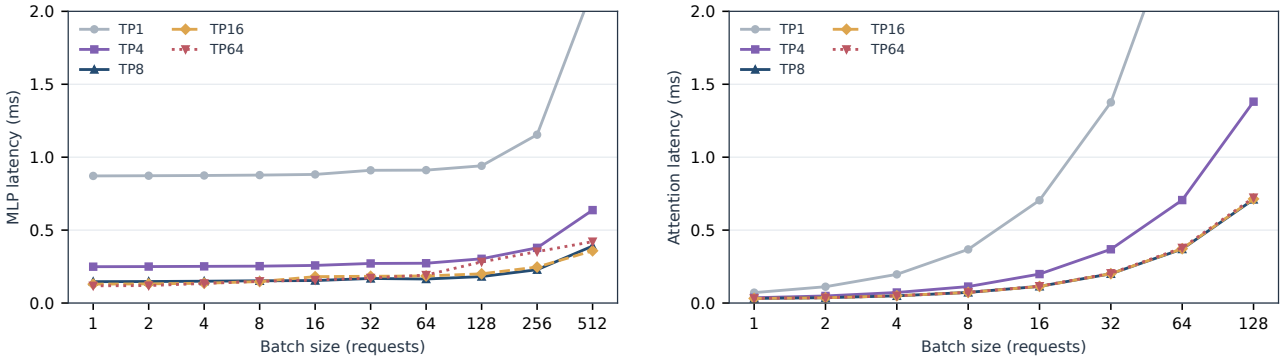
Inference engines combine multiple parallelism primitives to distribute computation and memory. Data parallelism increases throughput through model replicas, but repeated weight storage limits KV-cache capacity per GPU (Li et al., 2020; Kwon et al., 2023; Zheng et al., 2024). Tensor parallelism shards intra-layer computation and is effective within high-bandwidth interconnect domains (Shoeybi et al., 2020), while pipeline parallelism distributes consecutive layers across stages and overlaps their execution through micro-batching (Huang et al., 2019; Narayanan et al., 2019).

When the TP degree exceeds the KV-head count, standard TP replicates KV cache across ranks, limiting useful capacity and resident concurrency. Decode context parallel (DCP) works alongside TP to shard this cache along the sequence dimension instead, combining partial attention results through communication (Yang et al., 2025; Bhatia et al., 2025; Agrawal et al., 2025; Lin et al., 2024; Wu et al., 2024a; Liu et al., 2023). In current implementations, scaling DCP requires larger TP groups, which extend dense-layer collectives beyond NVLink when they span nodes. In contrast, TKNP retains intra-node TP for projections and MLPs and distributes attention across requests, so each owner computes complete attention outputs for its local head and requests with no cross-node merging of partial results.

Data-parallel attention (SGLang, 2024) is closest to TKNP's request-level ownership: each rank computes attention only for its own requests, avoiding KV replication and partial-result merging. However, every rank replicates the attention weights, and the dense layers must span the entire GPU pool, either by sharding weights across nodes with TP, as in Helix (Bhatia et al., 2025), which moves dense collectives off NVLink, or by replicating them, which reduces to DP for dense models. TKNP avoids both: only the active root executes each layer's dense operations, within its NVLink domain, while every node computes attention for its own requests. A single, balanced weight copy preserves KV capacity, and cross-node traffic reduces to point-to-point activation transfers sharded by request and head, each about $1/(NT)$ of the hidden state moved by a cross-node TP all-reduce for N nodes of T GPUs (§4.4).

3 MOTIVATION

High-throughput decoding requires both efficient execution and enough memory to keep active requests' KV cache resident. Using single-layer decode measurements and full-model memory budgets for Llama 3.1 405B on H100 GPUs, we show that dense layers and attention need different forms of parallelism (§3.1), and that KV capacity, not compute, limits resident concurrency (§3.2).



(a) MLP latency, including all-reduce.

(b) Attention latency with 32K context.

Figure 3. **Single-layer decode latency for Llama 3.1 405B** on H100 GPUs. (a) TP8 sustains relatively stable MLP latency through batch 128, with larger increases at batches 256–512. (b) Attention grows with batch size; TP8, TP16, and TP64 nearly overlap. Both panels use a 0–2 ms latency range; TP1 values above this range are clipped.

3.1 Dense computation and attention need different scaling

Increasing concurrency changes dense and attention costs differently. Figure 3 shows that at TP8 and 32K cached context, increasing the batch from 1 to 128 raises MLP latency by only 1.24× but attention latency by 22×. Attention consequently becomes the dominant component (Figure 1). Prior work similarly observes that attention becomes increasingly important as batch size and sequence length grow (Shrestha et al., 2025b). The relatively flat MLP curve reflects the memory-bound regime, in which weight reads dominate latency and are amortized across the batch (Figure 3a).

TP accelerates attention within a node, but gains plateau once its degree reaches the model's eight KV heads. At batch 128, attention falls 7.6× from TP1 to TP8 (5.38 to 0.71 ms), but an additional 8× more GPUs at TP64 yields no further improvement (Figure 3b). Replicating KV heads therefore adds GPUs without proportionally increasing attention bandwidth.

Dense computation encounters a different limit when all-reduces cross the inter-node network. Figure 4 shows that at batch 128, one all-reduce costs 6.5× more at TP64 than at TP8. Despite 8× less local compute per GPU, end-to-end MLP latency rises by 55%. DCP inherits this cost, because current implementations expand the TP group across nodes to increase DCP degree. These results favor retaining dense TP within NVLink while expanding attention along another dimension.

PP distributes weights without replication, but does not parallelize attention within a layer. Its microbatches also repeat dense weight reads: splitting batch 128 into four microbatches of 32 invokes each MLP four times. Because MLP latency is nearly flat, four batch-32 invocations cost **3.17**× one batch-128 invocation; pipeline overlap can hide part of this cost but cannot remove the repeated reads. Efficient decode scaling must therefore account for the different execution and communication requirements of attention and dense operations.

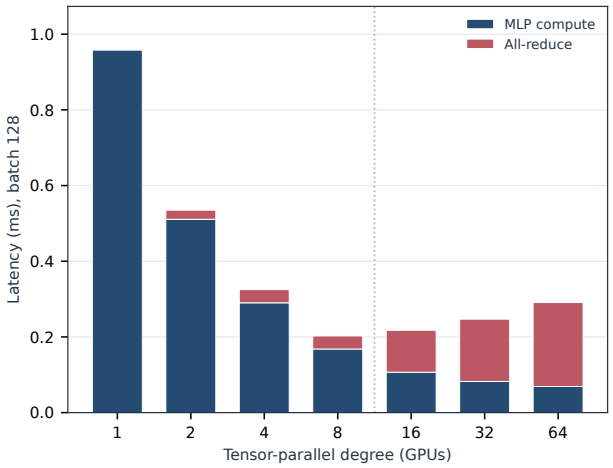


Figure 4. **Single-layer MLP compute and communication for Llama 3.1 405B** at batch 128. Bars sum local compute measurements and one isolated all-reduce. The dotted line separates single-node and multi-node TP.

3.2 Memory efficiency determines feasible concurrency and throughput

Memory capacity constrains the concurrency that fast decode kernels can sustain. Using FP8 quantization with selected parameters retained in BF16, Llama 3.1 405B model allocations on one eight-GPU TP8 node consume 454 GiB, leaving only 106 GiB for KV cache under the measured runtime budget. Yet batch 128 at 32K context requires

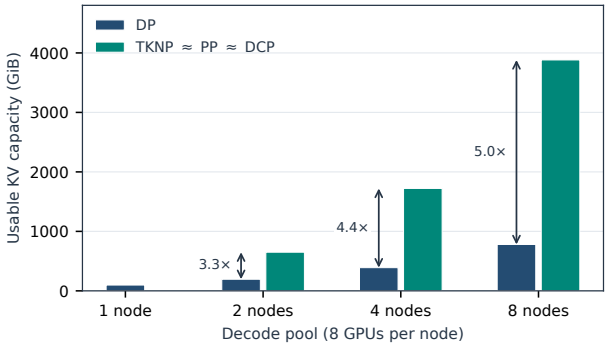


Figure 5. **Usable KV capacity for Llama 3.1 405B.** Each node contains eight GPUs; DP1 is TP8. TKNP, PP, and DCP provide approximately equal capacity, reaching 5.0× that of DP on eight nodes.

2,016 GiB of BF16 KV cache before output growth: 19× a node's KV budget, and 2.4× the combined budget of eight DP replicas.

As Figure 5 shows, memory-efficient layouts such as PP, DCP, and TKNP provide approximately 5.0× the usable KV capacity of DP on eight nodes by distributing model weights rather than storing a full model on every node. Standard TP instead spends part of this capacity on replicated KV heads.

Continuous batching cannot eliminate this capacity constraint: requests that do not fit remain queued, while smaller active batches limit weight-read amortization (Kwon et al., 2023). Efficient scaling therefore requires both sufficient KV capacity and an execution strategy that turns it into higher throughput. These observations motivate TKNP's combination of intra-node dense TP, request-parallel attention, and balanced weight placement, allowing the decode pool to expand attention bandwidth while preserving memory for KV cache.

4 Token Parallelism

Decode Token Parallelism separates layer-wise weight placement from request-local attention: dense operations for each layer execute within one node's TP group, while all nodes contribute attention bandwidth and KV capacity at every layer. The base design distributes one model copy across the pool (§4.1–§4.4); dual-active-root microbatching adds a second distributed copy to overlap execution (§4.5).

4.1 Architecture and Ownership

The decode pool contains N nodes of T GPUs, with TP within each NVLink domain and token-parallel communication between matching TP ranks across nodes. An external prefill server places the initial KV cache on the assigned decode owners in the required head layout before generation begins.

In single-copy execution, each node stores a contiguous block of layer weights and becomes the *active root* when that block executes. For layer ℓ, the active root $b(\ell)$ performs QKV projection, output projection, and MLP computation for the full batch using local TP. Independently, each *request owner* stores its assigned requests' KV cache for every layer and computes their attention, with heads sharded across its TP ranks. Attention therefore spans heads within nodes and requests across nodes, even when the layer's weights reside elsewhere (Figure 2). This separation is possible because attention over the KV cache is parameter-free: all of a layer's weights reside in its projections and MLP, so a request owner needs only the projected queries, keys, and values for the current token. We call this Token Parallelism because it distributes attention at the granularity of individual decode tokens, each associated with an active request and its locally owned KV cache.

4.2 Balanced Weight Placement

Separating weights and attention into dedicated pools introduces a capacity tradeoff (Chen et al., 2025; Xiang et al., 2026): including the weight-owning root in attention creates an uneven KV budget across nodes, while excluding it leaves its spare memory unavailable for KV cache. TKNP resolves this tradeoff by distributing weights across request-owning nodes, allowing the entire pool to contribute approximately equal KV capacity.

Layer-block weight placement. TKNP assigns contiguous layer blocks to different nodes and shards each block's weights across local TP ranks (Figure 6). Each node stores approximately $1/N$ of the model parameters. The block's node becomes the active root and executes projections and MLPs for the full batch, while every node computes attention for its assigned requests. This preserves full-batch weight-read amortization. The active-root role follows the blocks, while weights and KV cache remain stationary.

Capacity-aware request assignment. The scheduler assigns each waiting request to an eligible node with the lowest KV-cache load, measured by current usage or reserved context length. Eligibility requires sufficient capacity across the node's TP ranks for the input and expected output growth. If no node has enough capacity, the request remains queued. Once assigned, ownership stays fixed until completion, avoiding KV-cache migration. Because decode attention cost scales with resident KV length, balancing KV load also balances per-layer attention time.

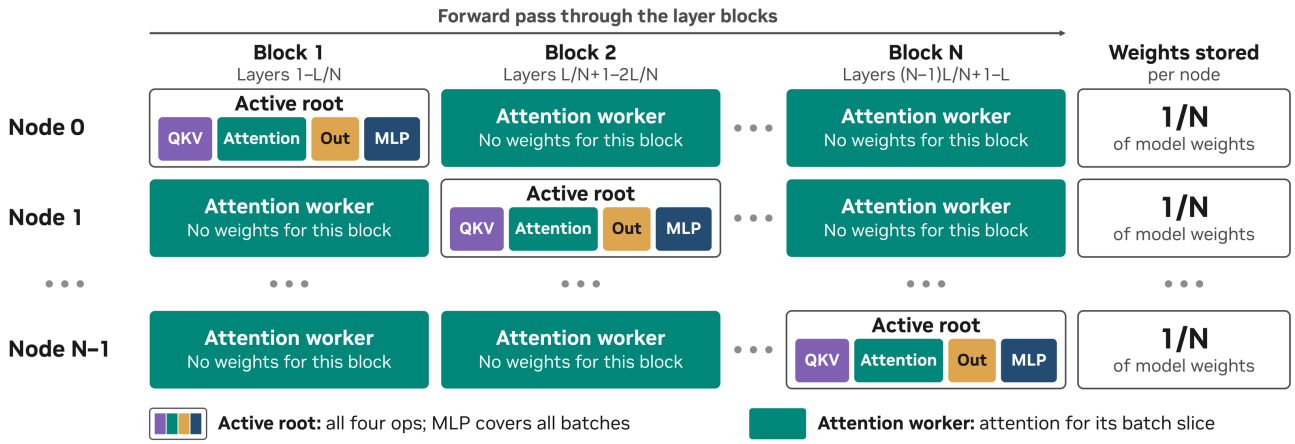


Figure 6. **Balanced weight placement and changing execution roles.** Each node stores one stationary layer block and becomes the active root when that block executes. Other nodes compute attention for their assigned requests. The schematic shows equal-sized blocks; actual boundaries balance weight bytes, including endpoint parameters and quantization differences, to leave similar KV-cache budgets.

4.3 Decode Execution and Communication

At each layer, the active root computes QKV projections for the full batch using local TP, then scatters the projected tensors to their request owners (Figure 2). All nodes, including root, apply rotary embeddings, append the new keys and values to local KV cache, and compute attention for their assigned requests and heads. The root gathers the attention outputs in batch order and completes the output projection, residual and normalization operations, and MLP. Each owner holds the complete KV cache for its heads, so attention requires no cross-node reduction of partial results, while dense all-reduces remain within NVLink.

Execution stays on the active root throughout its layer block. At each block boundary, hidden and residual activations pass to the next weight-owning node, which becomes the new root.

4.4 Token Parallel Communication Efficiency

TKNP confines cross-node attention exchange to peer-wise scatter/gather between matching TP ranks on the active root and request owners (Figure 2). Dense all-reduces stay within NVLink, and request-owned attention needs no cross-node aggregation of partial softmax results. This separates attention scaling from the cross-node collectives incurred when DCP expands the dense TP domain.

Let B be the active batch size, H_q and H_{kv} the query and KV-head counts, and d_h the head dimension. With intra-node TP degree T, each root rank holds projected QKV of shape

$$X^{\text{qkv}} \in \mathbb{R}^{B\times(H_q+2H_{kv})d_h/T}. \tag{1}$$

An owner assigned B_g requests receives and returns tensors of shapes

$$\begin{aligned} &\text{scatter:} \quad [B_g, (H_q+2H_{kv})d_h/T], \\ &\text{gather:} \quad [B_g, H_q/T, d_h]. \end{aligned} \tag{2}$$

Balanced assignment gives $B_g \approx B/N$; capacity-aware scheduling determines the actual split. The root's own requests require no network transfer. For s bytes per element, the combined scatter/gather payload per layer and matching-rank group is

$$C_\ell = \frac{2s(H_q+H_{kv})d_h}{T} \sum_{g\neq b(\ell)} B_g. \tag{3}$$

This counts each payload once, excluding padding, protocol overhead, and block handoffs. Individual transfers are sharded in both batch and head dimensions, and their volume is independent of context length.

For DCP degree N combined with TP degree NT, each rank computes partial attention over a sequence shard for the full batch. Its aggregation tensors have shapes

$$\begin{aligned} O_{\text{DCP}} &\in \mathbb{R}^{B\times H_q/T\times d_h}, \\ L_{\text{DCP}} &\in \mathbb{R}^{B\times H_q/T}, \end{aligned} \tag{4}$$

where L_{DCP} contains FP32 log-sum-exp statistics. DCP all-gathers these statistics and reduce-scatters the corrected partial outputs, whereas TKNP gathers complete outputs for only B_g requests. Under balanced assignment, the output tensor-size ratio is

$$\frac{|O_{\text{TKNP},g}|}{|O_{\text{DCP}}|} = \frac{B_g}{B} \approx \frac{1}{N}. \tag{5}$$

For Llama 3.1 405B at batch 128 on four TP8 nodes, BF16 partial outputs occupy 512 KiB per DCP rank, compared

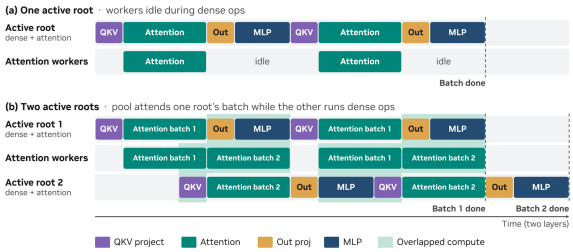


Figure 7. **Dual-active-root microbatching.** Shared workers alternate attention between two microbatches to overlap it with dense computation on the other root.

with 128 KiB per remote TKNP owner, a 75% reduction in output tensor size. DCP additionally exchanges 8 KiB of LSE statistics per rank; TKNP instead dispatches 144 KiB of QKV per remote owner. The N-fold reduction concerns individual output tensors, not total network traffic: DCP also gathers queries, and collective algorithms determine transfer volume.

By comparison, dense TP all-reduces operate on full-batch hidden activations of shape $[B, D]$, where D is the hidden dimension. Since $H_q d_h = D$ for both evaluated models, each TKNP gather is $1/(NT)$ of this operand: 128 KiB versus 4 MiB on four TP8 nodes. When TP spans nodes alongside DCP, these collectives traverse the inter-node network in addition to communication for sequence-sharded attention. TKNP instead keeps dense collectives local and exchanges projected head-space tensors across nodes, preserving the dense scaling benefits measured in Figure 4.

4.5 Dual-Active-Root Microbatching

Although attention dominates long-context decoding, attention workers still encounter brief idle gaps while the active root executes projections and the MLP. In larger decode pools, such as those with eight or more nodes, the additional KV capacity can make partial weight replication a favorable tradeoff, enabling greater compute overlap at the cost of some KV-cache capacity. For this regime, we propose splitting the batch into two independent microbatches served by distinct active roots. As shown in Figure 7, shared workers alternate attention between the two microbatches, using one microbatch's attention to fill gaps during the other's dense computation. This overlap offers an opportunity to improve throughput when the additional weight storage does not constrain request admission.

For even N, block b is stored on nodes b and $(b+N/2) \bmod N$, providing two distinct roots for each block. Every node stores two blocks, approximately $2/N$ of a model's weights instead of $1/N$, with endpoint parameters included in the balancing budget. This shifted placement distributes the second copy across the pool while keeping weights stationary as root roles rotate. Each request belongs to one microbatch and keeps a single owner for all layers. While a node serves as one microbatch's active root, it computes attention only for that microbatch's requests, so the two roots never contend for dense and attention work in the same phase. KV cache is neither migrated nor replicated.

5 Evaluation

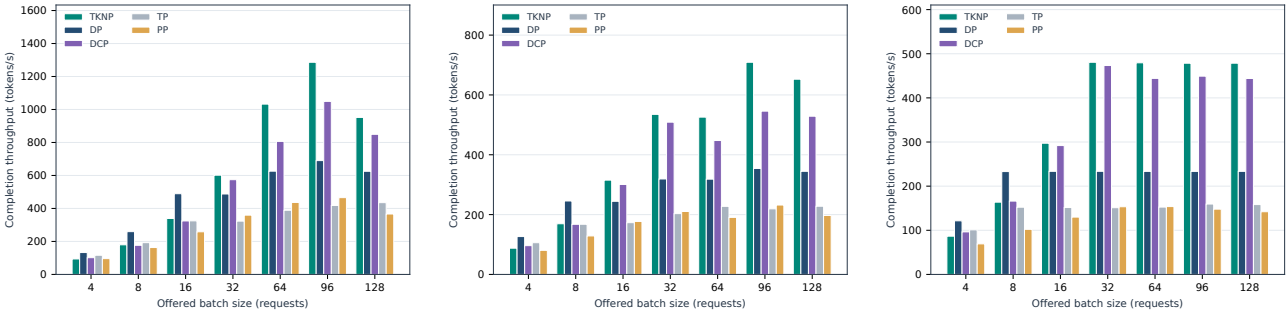
We integrate TKNP on top of vLLM by extending model construction and weight loading to instantiate dense parameters only on their assigned nodes, while retaining attention and local KV cache on every node. The scheduler tracks request ownership and per-owner KV budgets; the model runner maps scheduled requests into owner-specific execution slots and prepares local attention metadata. Dedicated communication groups connect matching TP ranks across nodes for scatter/gather and block handoffs, while reusing existing attention kernels and intra-node TP. Full CUDA-graph replay covers this distributed decode path through captured ownership layouts.

We evaluate how TKNP's combination of KV capacity and distributed attention affects decode throughput and request latency. The central question is when memory-efficient concurrency outweighs the execution advantage of independent replicas.

5.1 Experimental Setup

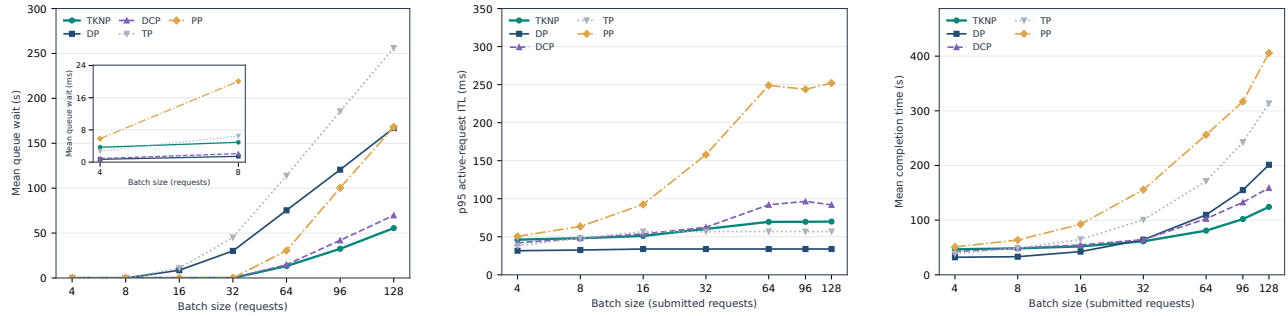
Disaggregated serving configuration. We consider a prefill–decode disaggregated setup with two prefill nodes and a decode pool of four or eight nodes. Each node contains 8×80GB H100 GPUs connected via 4th-generation NVLink and dual Intel Sapphire Rapids CPUs, with nodes interconnected by 8×400 Gb/s InfiniBand. We evaluate Llama 3.1 405B and Nemotron-4 340B using FP8 quantization with selected parameters retained in BF16, BF16 KV cache, and CUDA graphs on top of vLLM. Nemotron uses linear RoPE scaling beyond its original context window; these results characterize decode performance rather than long-context accuracy. Our software stack includes PyTorch 2.9.1, CUDA 12.9, NCCL 2.27.5, vLLM 0.19.0, and FlashAttention-3 (Shah et al., 2024), running on Linux kernel 5.15.0-88.

Parallel configurations. All experiments use the same two-node prefill configuration with TP8×DP2, isolating performance differences to the decode configuration. For N decode nodes, TKNP uses TP8 within each node and distributes weights and request-owned KV across the pool. DP uses N independent TP8 replicas; TP spans all $8N$ GPUs; DCP combines TP$8N$ with DCPN, since vLLM's DCP reuses the TP group; and PP uses TP8 within each of N stages. PP divides the resident batch into approximately N microbatches. All comparisons use matched decode GPU



(a) 32K context. (b) 64K context. (c) 96K context.

Figure 8. **Llama 3.1 405B completion throughput on 32 H100 GPUs**, including admission, capacity-induced waves, and drain, with 1,024 output tokens per request. While DP leads at low concurrency, TKNP achieves up to 2.06× speedups on large batch, long sequence workloads.



(a) Mean queue wait. (b) p95 active-request ITL. (c) Mean request completion time.

Figure 9. **Admission waiting, active-request ITL, and average completion latency** for Llama 3.1 405B at 64K context on 32 H100 GPUs, with 1K OSL. The x-axis counts all submitted requests; only a capacity-limited subset may be active. (a) Release to first scheduling. (b) Token intervals after the first token, excluding initial queue wait. (c) Average request completion time including queueing.

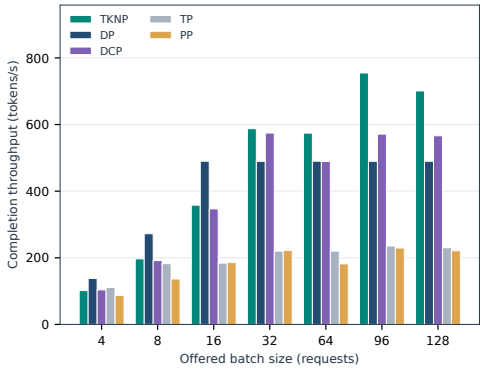


Figure 10. **Nemotron-4 340B completion throughput** at 64K context on 32 H100 GPUs, with 1,024 output tokens per request, including all admission waves.

counts and precision. Completion throughput and latency are measured at the decode pool from workload release; each request generates exactly the specified output length; and under greedy decoding, TKNP's generated tokens match those of the DP baseline.

Workload construction. We evaluate our system using two workload pools. First, we generate synthetic long-context workloads using RULER (Hsieh et al., 2024) with controlled input (ISL) and output sequence lengths (OSL) to study performance across varying concurrency and context lengths. For each context length, we hold the sequence length fixed while varying batch size, with each batch comprising distinct, non-identical requests. We disable prefix sharing across requests throughout our evaluation.

Second, we construct a realistic mixed-length workload from L-Eval, LongBench v1, and LongBench v2 (An et al., 2023; Bai et al., 2023; 2024), with sequence lengths ranging from 8,223 to 126,848 tokens (mean 32,898, median 19,048, standard deviation 28,496). The workload contains substantial variation in sequence length: 12.5% of requests exceed 64K tokens and contribute 36.1% of all input tokens, while the longest 25% account for 57.2%. This distribution captures heterogeneous KV-cache demand and attention computation across requests. Following prior work, we evaluate online serving under Poisson arrivals at varying request rates and a synchronized burst where all requests are submitted simultaneously (Zhong et al., 2024). All systems

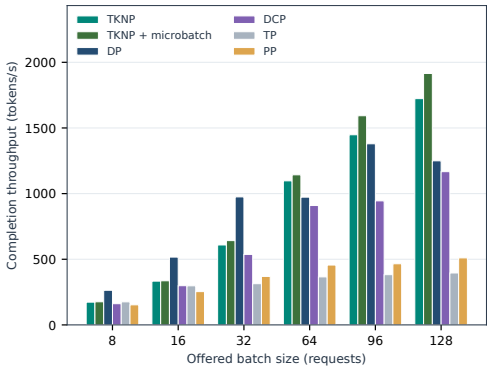


Figure 11. **Eight-node decode throughput with microbatching** for Llama 3.1 405B at 32K context on 64 H100 GPUs, with 1,024 output tokens per request. TKNP denotes the single-copy schedule; TKNP + microbatch uses two active roots.

use the same generated arrival trace at each request rate.

5.2 Performance across Concurrency and Context Length

We evaluate performance across varying concurrency levels and context lengths using fixed-length workloads, with each request generating 1,024 output tokens. We define completion throughput as the total number of generated output tokens divided by the elapsed time from workload release until the final request completes, including admission waiting, all execution waves, and final drain.

DP performs best when the submitted workload fits within per-replica KV-cache capacity. In this regime, replicas execute independently without TKNP's per-layer scatter/gather overhead, while small active batches provide limited attention work to parallelize. As concurrency increases, however, replicated model weights constrain DP's KV-cache capacity and reduce the number of requests that can execute concurrently. Excess requests are deferred to subsequent admission waves, limiting effective batch size and overall throughput. TKNP instead uses the distributed memory capacity to retain larger active batches while parallelizing attention across the decode pool. Consequently, the crossover from DP to TKNP occurs at lower concurrency as context length increases and KV capacity becomes constrained earlier (Figure 8). TKNP improves completion throughput by up to 2.06× over DP.

Comparisons with the remaining baselines further separate the effects of KV capacity and execution efficiency. TP replicates KV heads while introducing cross-node collectives for dense layers, limiting both capacity and execution efficiency. PP preserves aggregate KV capacity but partitions dense computation into smaller stage-local microbatches and confines each layer's attention computation to a single pipeline stage. DCP provides the most direct capacity-matched comparison: like TKNP, it shards the KV cache across the decode pool, but requires partial-attention aggregation and additional inter-node communication. TKNP therefore achieves its advantage over DCP with comparable KV capacity, indicating that the gains arise from more efficient distributed execution rather than additional cache capacity. On eight nodes, single-copy TKNP improves completion throughput by up to 1.53× over DCP; dual-active-root microbatching raises this advantage to 1.7× (Figure 11).

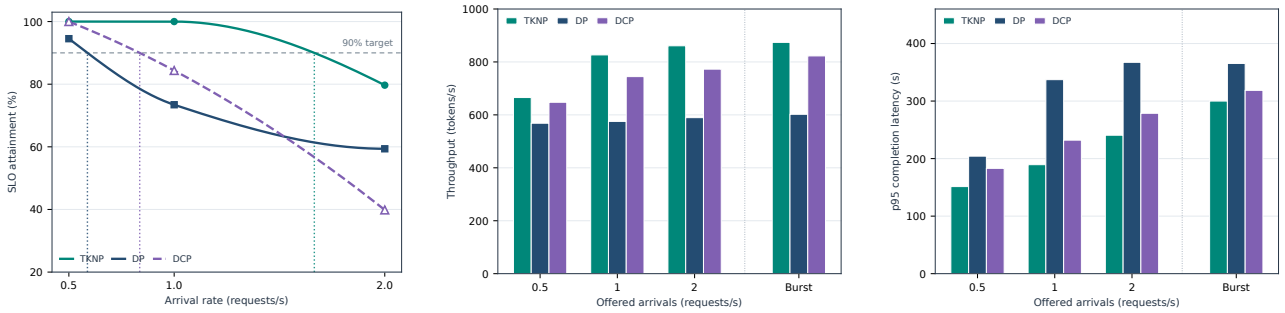
Nemotron-4 340B's smaller weight footprint leaves more KV capacity, improving DP's resident concurrency and competitiveness. TKNP nevertheless retains the high-concurrency advantage, achieving up to 1.54× higher throughput than DP and 1.32× than DCP (Figure 10).

Figure 9 separates the latency experienced by admitted requests from the end-to-end latency experienced by all submitted requests. Although the x-axis represents the full submitted batch, KV-cache constraints allow only a subset of requests to decode concurrently. Consequently, the lower active-request ITL of DP and TP in Figure 9b partly reflects their smaller resident batches and excludes the admission delay experienced by queued requests. Mean completion time (Figure 9c) captures both effects by measuring from request release through completion. At high concurrency, TKNP's greater resident capacity substantially reduces queuing and completion time even when DP achieves lower ITL for requests already admitted. Capacity alone, however, does not guarantee lower latency: PP admits larger batches than DP but can exhibit similar queueing delay because each wave takes longer to complete. TKNP combines greater resident capacity with efficient distributed execution for improving end-to-end request completion.

5.3 Scaling and Microbatching

For the same 128-request, 32K workload, doubling the decode pool from four to eight nodes increases single-copy TKNP throughput by 1.81×, corresponding to 90.5% scaling efficiency, compared with 1.37× for DCP (Figures 8 and 11). These gains include reduced admission waves as well as faster execution. DP nearly doubles throughput as its aggregate KV capacity grows, becoming more competitive with DCP, whose attention aggregation and dense collectives span the larger inter-node group. By keeping dense collectives within each node and using inter-node scatter/gather for attention, TKNP retains higher throughput at large batches and scales more effectively than DCP.

Microbatching improves throughput by up to 11.1%, reaching 1.7× the throughput of DCP, with the largest gains at high concurrency. The overlap opportunity is limited to the brief gaps between attention phases, since attention already dominates execution. Additional microbatch scheduling and synchronization, smaller dense batches with less



(a) SLO attainment. (b) Throughput. (c) p95 request completion time.

Figure 12. **TKNP sustains its throughput advantage under mixed input lengths.** Llama 3.1 405B on 32 H100 GPUs. (a) SLO: arrival-to-completion time per output token $\leq$100 ms. (b,c) Completion throughput and p95 request completion time under Poisson arrivals and synchronized bursts.

weight-read amortization, and reduced KV capacity from the second weight copy can further offset the benefit. These tradeoffs favor single-copy TKNP in smaller pools where memory limits resident concurrency.

5.4 Mixed-Length Serving Workloads

We next evaluate whether TKNP maintains its efficiency under the heterogeneous mixed-length workload described in Section 5.1. Despite substantial variation in per-request KV demand and attention work, TKNP achieves the highest throughput at every arrival rate, preserving the trends observed with fixed-length workloads (Figure 12b). This demonstrates that TKNP remains effective under realistic variation in request lengths.

The same behavior persists under staggered arrivals, where TKNP sustains high throughput while maintaining strong tail completion latency (Figure 12c). We define the completion SLO as arrival-to-completion latency normalized by the number of generated output tokens, including queueing time. At a 100 ms/output-token SLO, TKNP achieves full attainment at one request/s, while DP and DCP fall below 90% (Figure 12a). At two requests/s, TKNP retains higher SLO attainment than both baselines alongside its aggregate throughput advantage. Overall, TKNP maintains its efficiency across both synchronized and staggered arrivals, showing that its benefits generalize to heterogeneous serving workloads.

6 Limitations and Future Work

TKNP targets dense-model serving under concurrent, long-context decoding. At small batches or short contexts, communication and synchronization can outweigh the benefit of distributed attention. For the targeted workloads, intra-node TP provides effective scaling for projections and MLP computation, while TKNP extends attention bandwidth and KV capacity across nodes. Finally, our evaluation uses H100 nodes connected by InfiniBand; larger NVLink domains shift the balance between intra- and inter-node communication.

Several directions could extend TKNP. Applying TKNP's balanced layer-block placement to DCP would keep dense collectives within each node while sharding the sequence, removing DCP's need for cross-node TP and supporting requests whose KV cache exceeds one node. Combining TKNP with expert parallelism could reduce the attention-weight replication of data-parallel attention in mixture-of-experts serving. Dual-active-root microbatching improves throughput by up to 11.1% but adds scheduling, synchronization, and placement complexity. Adaptive schedulers that jointly account for KV occupancy and attention cost could capture this overlap with less memory overhead.

7 Conclusion

Decode Token Parallelism builds on the observation that dense layers and attention scale differently during decode. It decouples weight placement from attention ownership: each layer's dense operations run on a single node within its NVLink domain, while every node computes request-local attention against its own KV cache, and a single, balanced weight copy preserves KV capacity across the pool. On Llama 3.1 405B and Nemotron-4 340B, TKNP improves completion throughput by up to 1.7$\times$ over DCP and by up to 2.06$\times$ over DP. Dual-active-root microbatching further improves base TKNP throughput by up to 11.1% at larger decode pools through compute–attention overlap. Overall, TKNP demonstrates that decoupling attention scaling from model-weight placement enables both memory-efficient and high-throughput long-context decoding at scale.

REFERENCES

Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., Tumanov, A., and Ramjee, R. Taming throughput-latency tradeoff in llm inference with sarathi-serve. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*, OSDI'24, USA, 2024. USENIX Association. ISBN 978-1-939133-40-3.

Agrawal, A., Qiu, H., Chen, J., Íñigo Goiri, Zhang, C., Shahid, R., Ramjee, R., Tumanov, A., and Choukse, E. No request left behind: Tackling heterogeneity in long-context llm inference with medha, 2025. URL https://arxiv.org/abs/2409.17264.

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 4895–4901, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.298. URL https://aclanthology.org/2023.emnlp-main.298/.

An, C., Gong, S., Zhong, M., Zhao, X., Li, M., Zhang, J., Kong, L., and Qiu, X. L-eval: Instituting standardized evaluation for long context language models, 2023. URL https://arxiv.org/abs/2307.11088.

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., and Li, J. Longbench: A bilingual, multitask benchmark for long context understanding, 2023. URL https://arxiv.org/abs/2308.14508.

Bai, Y., Tu, S., Zhang, J., Peng, H., Wang, X., Lv, X., Cao, S., Xu, J., Hou, L., Dong, Y., Tang, J., and Li, J. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks, 2024. URL https://arxiv.org/abs/2412.15204.

Bhatia, N., More, A., Borkar, R., Mitra, T., Matas, R., Zhao, R., Golub, M., Mudigere, D., Pharris, B., and Rouhani, B. D. Helix parallelism: Rethinking sharding strategies for interactive multi-million-token llm decoding, 2025. URL https://arxiv.org/abs/2507.07120.

Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling, 2023. URL https://arxiv.org/abs/2302.01318.

Chen, S., Xiao, W., Lin, Y., Zhang, M., Shan, Y., Jiang, J., Chen, K., and Wu, Y. Efficient heterogeneous large language model decoding with model-attention disaggregation, 2025. URL https://arxiv.org/abs/2405.01814.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: fast and memory-efficient exact attention with io-awareness. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

Gautam, A., Shrestha, S., and Reddy, N. Token-driven gammatune: Adaptive calibration for enhanced speculative decoding, 2025. URL https://arxiv.org/abs/2504.00030.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Roziere, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Wyatt, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Guzmán, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Thattai, G., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I., Misra, I., Evtimov, I., Zhang, J., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., van der Linde, J., Billock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J., Alwala, K. V., Prasad, K., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., El-Arini, K., Iyer, K., Malik, K., Chiu, K., Bhalla, K., Lakhotia, K., Rantala-Yeary, L., van der Maaten, L., Chen, L., Tan, L., Jenkins, L., Martin, L., Madaan, L., Malo, L., Blecher, L., Landzaat, L., de Oliveira, L., Muzzi, M., Pasupuleti, M., Singh, M., Paluri, M., Kardas, M., Tsimpoukelli, M., Oldham, M., Rita, M., Pavlova, M., Kambadur, M., Lewis, M., Si, M., Singh, M. K., Hassan, M., Goyal, N., Torabi, N., Bashlykov, N., Bogoychev, N., Chatterji, N., Zhang, N., Duchenne, O., Çelebi, O., Alrassy, P., Zhang, P., Li, P., Vasic, P., Weng, P., Bhargava, P., Dubal, P., Krishnan, P., Koura, P. S., Xu, P., He, Q., Dong, Q., Srinivasan, R., Ganapathy, R., Calderer, R., Cabral, R. S., Stojnic,

R., Raileanu, R., Maheswari, R., Girdhar, R., Patel, R., Sauvestre, R., Polidoro, R., Sumbaly, R., Taylor, R., Silva, R., Hou, R., Wang, R., Hosseini, S., Chennabasappa, S., Singh, S., Bell, S., Kim, S. S., Edunov, S., Nie, S., Narang, S., Raparthy, S., Shen, S., Wan, S., Bhosale, S., Zhang, S., Vandenhende, S., Batra, S., Whitman, S., Sootla, S., Collot, S., Gururangan, S., Borodinsky, S., Herman, T., Fowler, T., Sheasha, T., Georgiou, T., Scialom, T., Speckbacher, T., Mihaylov, T., Xiao, T., Karn, U., Goswami, V., Gupta, V., Ramanathan, V., Kerkez, V., Gonguet, V., Do, V., Vogeti, V., Albiero, V., Petrovic, V., Chu, W., Xiong, W., Fu, W., Meers, W., Martinet, X., Wang, X., Wang, X., Tan, X. E., Xia, X., Xie, X., Jia, X., Wang, X., Goldschlag, Y., Gaur, Y., Babaei, Y., Wen, Y., Song, Y., Zhang, Y., Li, Y., Mao, Y., Coudert, Z. D., Yan, Z., Chen, Z., Papakipos, Z., Singh, A., Srivastava, A., Jain, A., Kelsey, A., Shajnfeld, A., Gangidi, A., Victoria, A., Goldstand, A., Menon, A., Sharma, A., Boesenberg, A., Baevski, A., Feinstein, A., Kallet, A., Sangani, A., Teo, A., Yunus, A., Lupu, A., Alvarado, A., Caples, A., Gu, A., Ho, A., Poulton, A., Ryan, A., Ramchandani, A., Dong, A., Franco, A., Goyal, A., Saraf, A., Chowdhury, A., Gabriel, A., Bharambe, A., Eisenman, A., Yazdan, A., James, B., Maurer, B., Leonhardi, B., Huang, B., Loyd, B., Paola, B. D., Paranjape, B., Liu, B., Wu, B., Ni, B., Hancock, B., Wasti, B., Spence, B., Stojkovic, B., Gamido, B., Montalvo, B., Parker, C., Burton, C., Mejia, C., Liu, C., Wang, C., Kim, C., Zhou, C., Hu, C., Chu, C.-H., Cai, C., Tindal, C., Feichtenhofer, C., Gao, C., Civin, D., Beaty, D., Kreymer, D., Li, D., Adkins, D., Xu, D., Testuggine, D., David, D., Parikh, D., Liskovich, D., Foss, D., Wang, D., Le, D., Holland, D., Dowling, E., Jamil, E., Montgomery, E., Presani, E., Hahn, E., Wood, E., Le, E.-T., Brinkman, E., Arcaute, E., Dunbar, E., Smothers, E., Sun, F., Kreuk, F., Tian, F., Kokkinos, F., Ozgenel, F., Caggioni, F., Kanayet, F., Seide, F., Florez, G. M., Schwarz, G., Badeer, G., Swee, G., Halpern, G., Herman, G., Sizov, G., Guangyi, Zhang, Lakshminarayanan, G., Inan, H., Shojanazeri, H., Zou, H., Wang, H., Zha, H., Habeeb, H., Rudolph, H., Suk, H., Aspegren, H., Goldman, H., Zhan, H., Damlaj, I., Molybog, I., Tufanov, I., Leontiadis, I., Veliche, I.-E., Gat, I., Weissman, J., Geboski, J., Kohli, J., Lam, J., Asher, J., Gaya, J.-B., Marcus, J., Tang, J., Chan, J., Zhen, J., Reizenstein, J., Teboul, J., Zhong, J., Jin, J., Yang, J., Cummings, J., Carvill, J., Shepard, J., McPhie, J., Torres, J., Ginsburg, J., Wang, J., Wu, K., U, K. H., Saxena, K., Khandelwal, K., Zand, K., Matosich, K., Veeraraghavan, K., Michelena, K., Li, K., Jagadeesh, K., Huang, K., Chawla, K., Huang, K., Chen, L., Garg, L., A, L., Silva, L., Bell, L., Zhang, L., Guo, L., Yu, L., Moshkovich, L., Wehrstedt, L., Khabsa, M., Avalani, M., Bhatt, M., Mankus, M., Hasson, M., Lennie, M., Reso, M., Groshev, M., Naumov, M., Lathi, M., Keneally, M., Liu, M., Seltzer, M. L., Valko, M., Restrepo, M., Patel, M., Vyatskov, M., Samvelyan, M., Clark, M., Macey, M., Wang, M., Hermoso, M. J., Metanat, M., Rastegari, M., Bansal, M., Santhanam, N., Parks, N., White, N., Bawa, N., Singhal, N., Egebo, N., Usunier, N., Mehta, N., Laptev, N. P., Dong, N., Cheng, N., Chernoguz, O., Hart, O., Salpekar, O., Kalinli, O., Kent, P., Parekh, P., Saab, P., Balaji, P., Rittner, P., Bontrager, P., Roux, P., Dollar, P., Zvyagina, P., Ratanchandani, P., Yuvraj, P., Liang, Q., Alao, R., Rodriguez, R., Ayub, R., Murthy, R., Nayani, R., Mitra, R., Parthasarathy, R., Li, R., Hogan, R., Battey, R., Wang, R., Howes, R., Rinott, R., Mehta, S., Siby, S., Bondu, S. J., Datta, S., Chugh, S., Hunt, S., Dhillon, S., Sidorov, S., Pan, S., Mahajan, S., Verma, S., Yamamoto, S., Ramaswamy, S., Lindsay, S., Lindsay, S., Feng, S., Lin, S., Zha, S. C., Patil, S., Shankar, S., Zhang, S., Zhang, S., Wang, S., Agarwal, S., Sajuyigbe, S., Chintala, S., Max, S., Chen, S., Kehoe, S., Satterfield, S., Govindaprasad, S., Gupta, S., Deng, S., Cho, S., Virk, S., Subramanian, S., Choudhury, S., Goldman, S., Remez, T., Glaser, T., Best, T., Koehler, T., Robinson, T., Li, T., Zhang, T., Matthews, T., Chou, T., Shaked, T., Vontimitta, V., Ajayi, V., Montanez, V., Mohan, V., Kumar, V. S., Mangla, V., Ionescu, V., Poenaru, V., Mihailescu, V. T., Ivanov, V., Li, W., Wang, W., Jiang, W., Bouaziz, W., Constable, W., Tang, X., Wu, X., Wang, X., Wu, X., Gao, X., Kleinman, Y., Chen, Y., Hu, Y., Jia, Y., Qi, Y., Li, Y., Zhang, Y., Zhang, Y., Adi, Y., Nam, Y., Yu, Wang, Zhao, Y., Hao, Y., Qian, Y., Li, Y., He, Y., Rait, Z., DeVito, Z., Rosnbrick, Z., Wen, Z., Yang, Z., Zhao, Z., and Ma, Z. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Hsieh, C.-P., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., and Ginsburg, B. Ruler: What's the real context size of your long-context language models?, 2024. URL https://arxiv.org/abs/2404.06654.

Hu, C., Huang, H., Xu, L., Chen, X., Wang, C., Xu, J., Chen, S., Feng, H., Wang, S., Bao, Y., Sun, N., and Shan, Y. Shuffleinfer: Disaggregate llm inference for mixed downstream workloads. *ACM Trans. Archit. Code Optim.*, 22 (2), July 2025. ISSN 1544-3566. doi: 10.1145/3732941. URL https://doi.org/10.1145/3732941.

Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, M. X., Chen, D., Lee, H., Ngiam, J., Le, Q. V., Wu, Y., and Chen, Z. *GPipe: efficient training of giant neural networks using pipeline parallelism*. Curran Associates Inc., Red Hook, NY, USA, 2019.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP

'23, pp. 611–626, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL https://doi.org/10.1145/3600006.3613165.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

Li, S., Zhao, Y., Varma, R., Salpekar, O., Noordhuis, P., Li, T., Paszke, A., Smith, J., Vaughan, B., Damania, P., and Chintala, S. Pytorch distributed: experiences on accelerating data parallel training. *Proc. VLDB Endow.*, 13(12):3005–3018, August 2020. ISSN 2150-8097. doi: 10.14778/3415478.3415530. URL https://doi.org/10.14778/3415478.3415530.

Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle-3: Scaling up inference acceleration of large language models via training-time test, 2025. URL https://arxiv.org/abs/2503.01840.

Lin, B., Zhang, C., Peng, T., Zhao, H., Xiao, W., Sun, M., Liu, A., Zhang, Z., Li, L., Qiu, X., Li, S., Ji, Z., Xie, T., Li, Y., and Lin, W. Infinite-llm: Efficient llm service for long context with distattention and distributed kvcache, 2024. URL https://arxiv.org/abs/2401.02669.

Liu, H., Zaharia, M., and Abbeel, P. Ring attention with blockwise transformers for near-infinite context, 2023. URL https://arxiv.org/abs/2310.01889.

Mitra, T., Borkar, R., Bhatia, N., Matas, R., Raj, S., Mudigere, D., Zhao, R., Golub, M., Dutta, A., Madduri, S., Jani, D., Pharris, B., and Rouhani, B. D. Beyond the buzz: A pragmatic take on inference disaggregation, 2025. URL https://arxiv.org/abs/2506.05508.

Mo, Z., Liao, J., Xu, H., Zhou, Z., and Xu, C. Hetis: Serving llms in heterogeneous gpu clusters with fine-grained and dynamic parallelism. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '25, pp. 1710–1724, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400714665. doi: 10.1145/3712285.3759784. URL https://doi.org/10.1145/3712285.3759784.

Narayanan, D., Harlap, A., Phanishayee, A., Seshadri, V., Devanur, N. R., Ganger, G. R., Gibbons, P. B., and Zaharia, M. Pipedream: generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP '19, pp. 1–15, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450368735. doi: 10.1145/3341301.3359646. URL https://doi.org/10.1145/3341301.3359646.

Nvidia, :, Adler, B., Agarwal, N., Aithal, A., Anh, D. H., Bhattacharya, P., Brundyn, A., Casper, J., Catanzaro, B., Clay, S., Cohen, J., Das, S., Dattagupta, A., Delalleau, O., Derczynski, L., Dong, Y., Egert, D., Evans, E., Ficek, A., Fridman, D., Ghosh, S., Ginsburg, B., Gitman, I., Grzegorzek, T., Hero, R., Huang, J., Jawa, V., Jennings, J., Jhunjhunwala, A., Kamalu, J., Khan, S., Kuchaiev, O., LeGresley, P., Li, H., Liu, J., Liu, Z., Long, E., Mahabaleshwarkar, A. S., Majumdar, S., Maki, J., Martinez, M., de Melo, M. R., Moshkov, I., Narayanan, D., Narenthiran, S., Navarro, J., Nguyen, P., Nitski, O., Noroozi, V., Nutheti, G., Parisien, C., Parmar, J., Patwary, M., Pawelec, K., Ping, W., Prabhumoye, S., Roy, R., Saar, T., Sabavat, V. R. N., Satheesh, S., Scowcroft, J. P., Sewall, J., Shamis, P., Shen, G., Shoeybi, M., Sizer, D., Smelyanskiy, M., Soares, F., Sreedhar, M. N., Su, D., Subramanian, S., Sun, S., Toshniwal, S., Wang, H., Wang, Z., You, J., Zeng, J., Zhang, J., Zhang, J., Zhang, V., Zhang, Y., and Zhu, C. Nemotron-4 340b technical report, 2024. URL https://arxiv.org/abs/2406.11704.

Patel, P., Choukse, E., Zhang, C., Shah, A., Íñigo Goiri, Maleki, S., and Bianchini, R. Splitwise: Efficient generative llm inference using phase splitting, 2024. URL https://arxiv.org/abs/2311.18677.

SGLang. SGLang v0.4: Zero-overhead batch scheduler, cache-aware load balancer, faster structured outputs. https://lmsys.org/blog/2024-12-04-sglang-v0-4/, December 2024. LMSYS Org Blog. Accessed October 2026.

Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024. URL https://arxiv.org/abs/2407.08608.

Shazeer, N. Fast transformer decoding: One write-head is all you need, 2019. URL https://arxiv.org/abs/1911.02150.

Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020. URL https://arxiv.org/abs/1909.08053.

Shrestha, S., Reddy, N., and Li, Z. Espn: Memory-efficient multi-vector information retrieval. In *Proceedings of the 2024 ACM SIGPLAN International Symposium on Memory Management*, ISMM 2024, pp. 95–107, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400706158. doi: 10.1145/3652024.3665515. URL https://doi.org/10.1145/3652024.3665515.

Shrestha, S., Gautam, A., and Reddy, N. Storage access optimization for efficient gpu-centric information retrieval. *The Journal of Supercomputing*, 81(4):613, 2025a. doi: 10.1007/s11227-025-07118-9. URL https://doi.org/10.1007/s11227-025-07118-9.

Shrestha, S., Settlemyer, B., Dryden, N., and Reddy, N. Polar sparsity: High throughput batched llm inferencing with scalable contextual sparsity, 2025b. URL https://arxiv.org/abs/2505.14884.

Wu, B., Liu, S., Zhong, Y., Sun, P., Liu, X., and Jin, X. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, SOSP '24, pp. 640–654, New York, NY, USA, 2024a. Association for Computing Machinery. ISBN 9798400712517. doi: 10.1145/3694715.3695948. URL https://doi.org/10.1145/3694715.3695948.

Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., Huang, G., Liu, X., and Jin, X. Fast distributed inference serving for large language models, 2024b. URL https://arxiv.org/abs/2305.05920.

Xiang, G., Kang, X., Zhang, L., Lin, W., Shi, S., Wang, Y., and Chu, X. Kernelflume: Elastic core-attention scaling for agentic long-context decoding, 2026. URL https://arxiv.org/abs/2606.29207.

Yang, A., Yang, J., Ibrahim, A., Xie, X., Tang, B., Sizov, G., Reizenstein, J., Park, J., and Huang, J. Context parallelism for scalable million-token inference, 2025. URL https://arxiv.org/abs/2411.01783.

Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., and Ceze, L. Flashinfer: Efficient and customizable attention engine for llm inference serving, 2025. URL https://arxiv.org/abs/2501.01005.

Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pp. 521–538, Carlsbad, CA, July 2022. USENIX Association. ISBN 978-1-939133-28-1. URL https://www.usenix.org/conference/osdi22/presentation/yu.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. Sglang: efficient execution of structured language model programs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385.

Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. Distserve: disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*, OSDI'24, USA, 2024. USENIX Association. ISBN 978-1-939133-40-3.

Zhu, R., Jiang, Z., Jin, C., Wu, P., Stuardo, C. A., Wang, D., Zhang, X., Zhou, H., Wei, H., Cheng, Y., Xiao, J., Zhang, X., Liu, L., Lin, H., Chang, L.-W., Ye, J., Yu, X., Liu, X., Jin, X., and Liu, X. Megascale-infer: Efficient mixture-of-experts model serving with disaggregated expert parallelism. In *Proceedings of the ACM SIGCOMM 2025 Conference*, SIGCOMM '25, pp. 592–608, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400715242. doi: 10.1145/3718958.3750506. URL https://doi.org/10.1145/3718958.3750506.